

voice.py — установка и использование

Скрипт для озвучки сценариев через Gemini 3.8 Flash TTS. Команды даны для Windows (PowerShell), отличия для macOS и Linux указаны отдельно.

1. Что нужно

Компонент	Зачем	Проверка
Python 3.10+	Запуск скрипта	<code>python --version</code> (на macOS/Linux: <code>python3 --version</code>)
google-genai	Библиотека для Gemini API	Устанавливается одной командой (шаг 2)
API-ключ Gemini	Доступ к модели	Берётся в AI Studio (шаг 2)
FFmpeg	Только для ускорения реплик при <code>--fit</code> . Необязательно	<code>ffmpeg -version</code>

Если команда проверки выдаёт версию — компонент уже есть, этот пункт установки можно пропустить.

2. Установка

Python

Система	Команда
Windows	<code>winget install Python.Python.3.13</code> или установщик с python.org (отметь Add python.exe to PATH)
macOS	<code>brew install python</code>
Linux (Debian/Ubuntu)	<code>sudo apt install python3 python3-pip</code>

После установки закрой и заново открой терминал. На macOS и Linux дальше пиши `python3` и `pip3` вместо `python` и `pip`.

FFmpeg (необязательно, только для `--fit`)

Система	Команда
Windows	<code>winget install Gyan.FFmpeg</code>
macOS	<code>brew install ffmpeg</code>
Linux (Debian/Ubuntu)	<code>sudo apt install ffmpeg</code>

Без FFmpeg скрипт работает полностью, только `--fit` будет подгонять реплики одной регенерацией, без ускорения.

Библиотека

```
pip install -U google-genai
```

API-ключ

Создай ключ в AI Studio: **Get API key** (иконка ключа в левом нижнем углу).

Только для текущего окна PowerShell:

```
$env:GEMINI_API_KEY="твой_ключ"
```

Навсегда (выполнить один раз, затем переоткрыть PowerShell):

```
setx GEMINI_API_KEY "твой_ключ"
```

macOS / Linux (чтобы навсегда — добавь эту строку в `~/.zshrc` или `~/.bashrc`):

```
export GEMINI_API_KEY="твой_ключ"
```

Файлы

Скачай репозиторий и перейди в его папку:

```
git clone https://github.com/justajazz/VoiceOver.git
cd VoiceOver
```

Без git: на странице репозитория **Code** → **Download ZIP**, распакуй и перейди в папку командой `cd`. Путь к папке проще всего скопировать из адресной строки Проводника.

3. Настройка голосов

Открой `voice.py` в любом редакторе. В начале файла блок настроек:

```
MODEL = "gemini-3.8-flash-tts" # или "gemini-3.8-flash-lite-tts" (дешевле и быстрее)

VOICES = {
    "Alex": "Puck",
    "Max": "Charon",
}

OUTPUT_DIR = "output"
```

- Ключ ("**Alex**") — имя спикера в сценарии после @.
- Значение — встроенный голос (**Puck**, **Charon**, **Fola...**) или ID своего голоса из Voice Design (`voice_...`).
- Добавил спикера в сценарий — добавь его сюда, иначе скрипт остановится с ошибкой.

4. Режимы работы

Команда	Что делает	Результат
<code>python voice.py dialogue.txt</code>	Весь диалог одним запросом. Естественные переходы и бэкченнелы <code> ... </code> . До 2 спикеров, только встроенные голоса	<code>dialogue.wav</code>
<code>python voice.py dialogue.txt --lines</code>	Каждая реплика отдельным запросом. Работают свои голоса <code>voice_...</code> и любое число спикеров	<code>01_Alex.wav</code> , <code>02_Max.wav</code> ... + <code>full.wav</code>
<code>python voice.py timed.txt</code>	Сценарий с таймкодами: реплики раскладываются по времени	реплики + <code>timeline.wav</code> + <code>report.txt</code>
<code>python voice.py timed.txt --fit</code>	То же + подгонка реплик, которые не влезают в своё окно	то же, длинные реплики подогнаны

Режим с таймкодами включается автоматически, если таймкоды стоят у всех реплик. Флаг `--lines` в этом случае не нужен.

5. Параметры

Параметр	По умолчанию	Что делает
<code>--lines</code>	выкл.	Каждая реплика отдельным файлом + склейка <code>full.wav</code>
<code>--gap 0.5</code>	0.3	Пауза между репликами в <code>full.wav</code> , секунды
<code>--fit</code>	выкл.	Подгонка длинных реплик: сначала регенерация со style <code>speaking rapidly</code> , потом ускорение через FFmpeg
<code>--max-speed 1.15</code>	1.2	Максимальное ускорение FFmpeg. Больше 1.2 заметно на слух
<code>--retries 2</code>	1	Сколько раз регенерировать реплику быстрее (каждый раз — платный запрос)
<code>--fps 30</code>	25	Частота кадров для таймкодов вида <code>чч:мм:сс:кадры</code>
<code>--start 01:00:00:00</code>	—	Таймкод начала монтажки (в DaVinci по умолчанию 01:00:00:00)

6. Что получается на выходе

Каждый запуск создаёт новую папку, поэтому ничего не перезаписывается:

output\ 2026-09-26_18-30-12\ script.txt dialogue.wav 01_Alex.wav, 02_Max.wav full.wav 01_00m00.50s_Alex.wav timeline.wav report.txt	копия сценария этого запуска режим диалога режим --lines режим --lines, склейка режим таймкодов, реплика с временем в имени режим таймкодов, дорожка от 00:00 режим таймкодов, отчёт
---	--

Как читать report.txt

[01]	00:00.50	Alex	длина	5.50 с		окно	5.50 с		OK	ускорено x1.10
[02]	00:06.00	Max	длина	4.17 с		окно	3.50 с		НАЛЕЗАЕТ на следующую на 0.67 с	
[06]	00:23.00	Max	длина	5.79 с		окно	— с		OK	

- **длина** — сколько звучит реплика; **окно** — время до следующей реплики (у последней окна нет).
- **OK** — влезает. Пометка справа: что сделал **--fit** (перегенерировал или ускорил).
- **НАЛЕЗАЕТ** — реплика перекрывает следующую. В **timeline.wav** они будут звучать одновременно. Сократи текст или сдвинь таймкод.

7. Работа с видеорядом

1. В Premiere или DaVinci пройди по видео и выпиши таймкоды, где должна начинаться каждая реплика.
2. Запиши их в сценарий: **@00:12.5 Max: calm** (или в формате монтажки **@01:00:12:10 Max с --fps**).
3. Запусти **python voice.py timed.txt --fit** (для DaVinci добавь **--start 01:00:00:00**).
4. Проверь **report.txt**, исправь налезавшие реплики и перезапусти.
5. Положи **timeline.wav** в начало таймлайна — реплики совпадут с видео. Для ручной подстройки используй отдельные файлы реплик.

8. Ошибки и решения

Сообщение	Причина	Что делать
Не найден GEMINI_API_KEY	Ключ не задан в этом окне	<code>\$env:GEMINI_API_KEY="..."</code> или <code>setx</code> и переоткрыть PowerShell
python не является командой / command not found	Python не установлен или не в PATH	Установить Python (шаг 2), отметить Add to PATH, переоткрыть терминал. На macOS/Linux — <code>python3</code>
ModuleNotFoundError: google	Библиотека не установлена	<code>pip install -U google-genai</code>
Нет голоса для спикеров [...]	Имя из сценария отсутствует в VOICES	Добавить имя в VOICES (регистр важен)
В режиме диалога максимум 2 спикера	В сценарии больше двух имён	Запустить с <code>--lines</code>
Свои голоса (voice_...) в одном диалоге не поддерживаются	Ограничение API	Запустить с <code>--lines</code>
Таймкоды должны быть у всех реплик	У части блоков нет таймкода	Добавить таймкоды или убрать все
Таймкод ... раньше начала монтажки	Неверный <code>--start</code>	Проверить <code>--start</code> и таймкоды
ffmpeg не найден	FFmpeg не установлен или не в PATH	Установить FFmpeg (шаг 2) или запускать <code>--fit</code> без ускорения
429 RESOURCE_EXHAUSTED	Лимит запросов или потолок трат	Подождать; проверить Dashboard → Usage и Spend

9. Расходы

- Запросы к Gemini TTS платные. У каждого платного уровня есть системный потолок трат в месяц (для Tier 1 — \$250). Это максимум, а не сумма к оплате.
- Свой лимит на проект: AI Studio → вкладка **Spend** → **Monthly spend cap** (например, €20).
- `--fit` и `--retries` делают дополнительные запросы. Для экономии сначала запускай без `--fit` и смотри `report.txt`.
- `gemini-3.8-flash-lite-tts` в MODEL — дешевле и быстрее, качество немного ниже.